KernelMix reference manual

Nilson Chapagain

Contents

1 KernelMix **2**
1.1 Install . . . 2
1.2 Example . . . 3
1.3 Documentation . . . 4
1.4 References and license . . . 4

2 Model and interpretation **4**
2.1 Continuous outcomes . . . 4
2.2 Binary outcomes . . . 4
2.3 Count outcomes . . . 5
2.4 Survival outcomes . . . 5
2.5 Random intercepts . . . 6
2.6 Priors and exposure selection . . . 6
2.7 Inputs and computation . . . 7
2.8 Prediction and contrasts . . . 7
2.9 Diagnostics and scope . . . 8

3 An exposure-mixture analysis **8**
3.1 Define the data . . . 8
3.2 Fit and inspect a continuous outcome . . . 8
3.3 Predict at exposure profiles . . . 9
3.4 Summarize exposure associations . . . 9
3.5 Fit a binary outcome . . . 10
3.6 Fit counts and include observation time . . . 10
3.7 Fit right-censored survival times . . . 10
3.8 Account for clustered observations . . . 11
3.9 Plot the results . . . 12
3.10 Save and load a fit . . . 12

4 Structured models and GP approximations **12**
4.1 Choose an exposure kernel . . . 13
4.2 Reduce GP matrix size . . . 13
4.3 Model random intercepts and slopes . . . 14
4.4 Separate Gaussian variance and correlation . . . 15
4.5 Encode delayed entry and competing events . . . 16
4.6 Record left or interval censoring . . . 16
4.7 Supply known covariate paths . . . 17
4.8 Set starting values . . . 17

5 API reference **18**
5.1 Fitting . . . 18
5.2 Structured model results . . . 20
5.3 Fit results and diagnostics . . . 20
5.4 Prediction . . . 21

5.5 Cluster and survival results . . . 22
5.6 Exposure contrasts and curves . . . 22
5.7 Plotting . . . 24
5.8 Kernel helper . . . 24
5.9 Method sources . . . 24

6 Checks and development **24**
6.1 Run checks . . . 24
6.2 Numerical evidence . . . 25
6.3 Build the manual . . . 26

1 KernelMix

Bayesian kernel machine regression (BKMR) for exposure mixtures in Python. KernelMix estimates a flexible exposure–response function, with linear adjustment for covariates. It supports continuous, binary, count and right-censored survival outcomes, with random intercepts and slopes for clustered observations. NumPy and SciPy handle fitting and prediction; R is not required.

Analysis	Functions
Fit outcomes	`fit`, `Prior`: Gaussian, probit, Poisson, negative binomial and piecewise-exponential survival
Cluster observations	Random intercepts/slopes; Gaussian AR1, exponential or supplied residual correlations
Model unequal residual variance	Gaussian variance regression and residual-variance prediction
Choose a GP representation	Gaussian, Matérn 3/2 or rational-quadratic kernels; dense, predictive-process or random-Fourier bases
Select exposures or exposure groups	`selection="component"` or `"group"`, posterior inclusion probabilities
Inspect results	`summary`, `diagnostics`, `posterior_samples`
Predict	`predict_h`, `predict`, `predict_survival`, `posterior_moments_h`
Describe exposure associations	Overall, single-exposure and interaction contrasts; response curves and surfaces
Plot	Chains, inclusion probabilities, exposure curves, survival curves, contrasts and surfaces

1.1 Install

With access to this repository:

```
python -m pip install "kernelmix[plots] @ git+https://github.com/nilson01/kernelmix.git@main"
```

Or, from a downloaded source folder:

```
python -m pip install ".[plots]"
```

Python 3.10 or later is required. Omit `[plots]` if plots are not needed.

1.2 Example

```
import numpy as np
from kernelmix import fit, univariate_response, plot_response

rng = np.random.default_rng(17)
Z = rng.normal(size=(24, 2))
y = np.sin(Z[:, 0]) + 0.3 * Z[:, 1] + rng.normal(0, 0.5, 24)
result = fit(y, Z, draws=200, warmup=200, chains=2, seed=17)
print(result.summary())
curve = univariate_response(result, variable=0, points=25, seed=17)
ax = plot_response(curve)
```

Example output (first rows):

```
BKMRFit(family='gaussian', n=24, exposures=2, chains=2, draws=200)
95% pointwise credible intervals
Parameter             Mean         SD      Lower      Upper
lambda               16.93      9.047      4.145      38.61
sigma2              0.2159    0.08346     0.1142     0.4188
```

These short chains trigger mixing warnings. This example demonstrates the workflow. Choose chain lengths using mixing, effective sample size and the precision needed for the analysis.

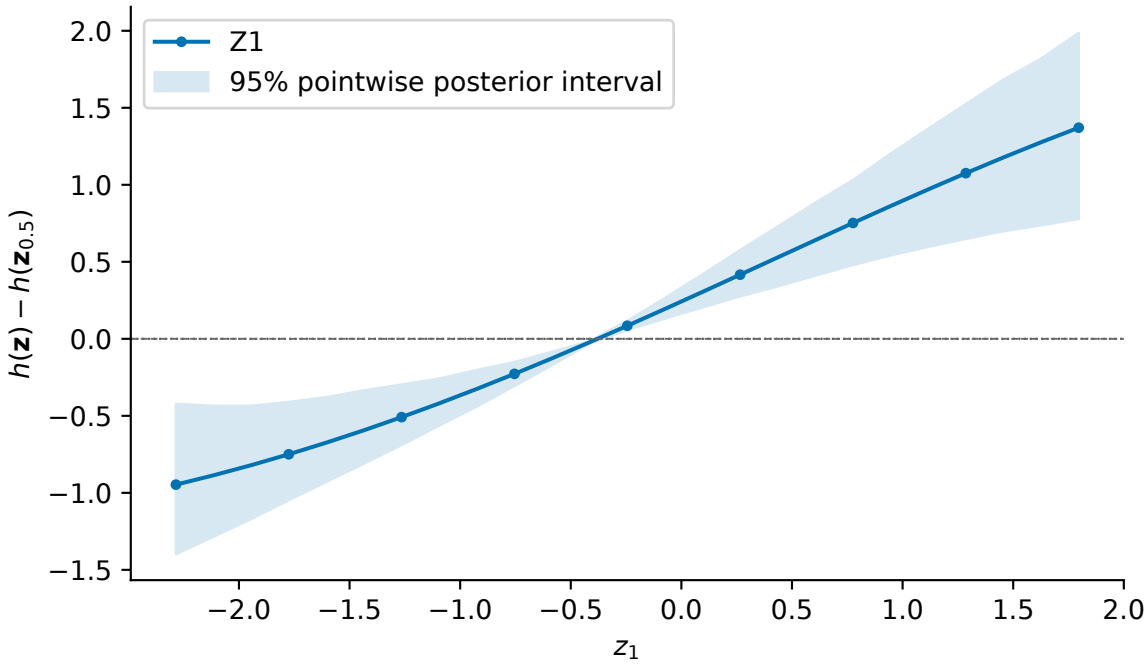


Figure 1: Exposure response curve with pointwise 95% credible intervals

The curve holds the second exposure at its sample median and subtracts the all-median reference. It shows an association, not an identified causal effect.

1.3 Documentation

- Tutorial: data, fitting, prediction and plots.
- Structured models: kernels, approximations, residual structures and extended survival.
- API: grouped functions, inputs and returned values.
- Model and interpretation: equations, priors and limits.
- Reference manual: printable documentation.
- Checks and development: tests and installed-R comparisons.

Dense covariance calculations limit practical sample size. The default kernel is Gaussian. Survival fitting uses a full event-time likelihood with a piecewise-constant baseline hazard. Structured survival inputs support delayed entry, competing causes, piecewise-constant predictor histories, and left or interval censoring under the stated observation assumptions. Group selection allows at most one selected exposure per group. Diagnostics cannot establish convergence or validate model assumptions.

1.4 References and license

The method is due to Bobb et al. (2015). The software and binary-outcome extension are described by Bobb et al. (2018). KernelMix is a separate Python implementation. Selected Gaussian/probit calculations, including random intercepts, are checked against R `bkmr` 0.2.2. See NOTICE for attribution and LICENSE for GPL-2.0-only terms.

Count and survival likelihoods use additional standard model ingredients, separately from R `bkmr`. Their definitions and checks are in the model guide. New-family default amplitude priors differ from the Gaussian/probit defaults.

2 Model and interpretation

2.1 Continuous outcomes

Following Bobb et al. (2015),

$$Y_i = h(\mathbf{z}_i) + \mathbf{x}_i^\mathsf{T}\boldsymbol{\beta} + \epsilon_i, \qquad \epsilon_i \overset{\text{iid}}{\sim} N(0, \sigma^2).$$

Here, Y_i is the outcome, $\mathbf{z}_i$ contains the M exposures, $\mathbf{x}_i$ contains adjustment covariates, and $\boldsymbol{\beta}$ contains their coefficients. The function h allows nonlinear exposure associations and interactions. Its prior at the observed exposure profiles is

$$\mathbf{h} = (h_1, \ldots, h_n)^\mathsf{T} \sim N(\mathbf{0}, \tau\mathbf{K}_{Z,\mathbf{r}}), \qquad K(\mathbf{z}, \mathbf{z}'; \mathbf{r}) = \exp\left\{-\sum_{m=1}^{M} r_m(z_m - z'_m)^2\right\}.$$

The relevance parameters $r_m \geq 0$ determine how each exposure changes the kernel. An exposure with $r_m = 0$ does not affect h. In the default marginalized Gaussian sampler, `lambda` is $\lambda = \tau/\sigma^2$. Integrating out $\mathbf{h}$ gives covariance $\sigma^2(\mathbf{I} + \lambda\mathbf{K}_{Z,\mathbf{r}})$.

2.2 Binary outcomes

With `family="binomial"`, the model uses a probit link:

$$\Pr(Y_i = 1 \mid h, \boldsymbol{\beta}) = \Phi\{h(\mathbf{z}_i) + \mathbf{x}_i^\mathsf{T}\boldsymbol{\beta}\}.$$

Here Φ is the standard normal cumulative distribution function. Residual variance on the latent normal scale is fixed at one. This extension follows Bobb et al. (2018). Probability contrasts depend on both exposure profiles and adjustment covariates. They are not odds ratios. The default sampler rejects separated linear adjustment designs because its flat coefficient prior can give an improper posterior. Structured fits use a proper normal coefficient prior.

2.3 Count outcomes

For Poisson and NB2 outcomes, define

$$\eta_i = h(\mathbf{z}_i) + \mathbf{x}_i^\top \boldsymbol{\beta} + u_{c(i)}, \qquad \mu_i = \exp(\eta_i + o_i).$$

The known log offset o_i accounts for observation time or exposure amount. Without clustering, $u_{c(i)} = 0$. The Poisson model has conditional mean and variance μ_i. The NB2 model has

$$E(Y_i \mid \mu_i, \phi) = \mu_i, \qquad \mathrm{Var}(Y_i \mid \mu_i, \phi) = \mu_i + \mu_i^2/\phi, \qquad \phi > 0.$$

`dispersion` stores ϕ. Larger values approach Poisson variation. These likelihoods follow the standard Poisson and NB2 parameterizations. They extend KernelMix beyond R `bkmr` 0.2.2; they are not described as R BKMR implementations or new statistical methods.

2.4 Survival outcomes

For positive observed time t_i and event indicator d_i, the model is

$$\lambda_i(t) = \lambda_0(t)\exp(\eta_i), \qquad S_i(t) = \exp\{-\Lambda_0(t)\exp(\eta_i)\}.$$

Here $\lambda_0(t)$ is the baseline hazard and $\Lambda_0(t)$ its integral. The linear predictor η_i is defined above without a count offset. Independent right censoring gives likelihood contribution $\{\lambda_i(t_i)\}^{d_i} S_i(t_i)$. The model uses constant baseline rates λ_{0j} within fixed, right-closed intervals. If D_{ij} is subject i's time in interval j, $j(i)$ contains its event time, and d_j counts events in interval j, then

$$\ell = \sum_i d_i\{\log \lambda_{0,j(i)} + \eta_i\} - \sum_i e^{\eta_i} \sum_j D_{ij}\lambda_{0j}.$$

An event at a cut belongs to the interval on its left. Every event then has positive time at risk in its event interval. With independent $\lambda_{0j} \sim \mathrm{Gamma}(a_0, b_0)$ priors, the conditional update is

$$\lambda_{0j} \mid \cdots \sim \mathrm{Gamma}\left(a_0 + d_j,\ b_0 + \sum_i D_{ij} e^{\eta_i}\right).$$

This is a full piecewise-exponential event-time likelihood, not Cox partial likelihood. See the INLA manual, section 3.15 for the model form and the Stan survival guide for censoring contributions. KernelMix uses its own independent gamma baseline prior. Cutpoints are conditioned on after selection; their uncertainty is not modeled. Defaults use event-time deciles, so assess sensitivity to their number and placement. Baseline priors depend on the chosen time unit.

The basic interface below describes right censoring. The structured survival interface also supports delayed entry, competing causes, predictor histories and left/interval censoring. All-censored data are accepted with a warning: the posterior then depends strongly on the priors. A fitted intercept and the baseline hazard share a scale. Separate estimates are anchored by their priors; `intercept=False` avoids adding that redundant coefficient.

2.5 Random intercepts

For the default random-intercept route, with observation-cluster index $c(i)$, independent Gaussian intercepts satisfy $u_g \sim N(0, \sigma_u^2)$. Exposures in `groups` and observations in `cluster` serve different purposes. Conditional on h, covariates and random intercepts, observations are independent under the selected outcome likelihood.

For Gaussian/probit fits, `lambda_cluster` is σ_u^2/σ^2, with probit $\sigma^2 = 1$. The marginalized covariance is

$$\sigma^2\{\mathbf{I} + \lambda \mathbf{K}_{Z,\mathbf{r}} + \lambda_u \mathbf{J}\}, \qquad J_{ij} = 1\{c(i) = c(j)\}.$$

For count/survival fits there is no residual Gaussian variance parameter: `lambda` and `lambda_cluster` are the GP and random-intercept variances directly. The random-intercept model follows the Gaussian/probit extension described by Bobb et al. (2018). Its use with the new likelihoods is an additional model composition.

Known-cluster prediction conditions on the cluster posterior. Equal unseen labels share a new Gaussian draw. Omitted labels fix $u = 0$; this is not marginalization over the cluster distribution. Random intercepts do not provide arbitrary serial correlation or cluster-robust inference. GP and cluster variance components can be weakly separated, particularly for constant kernels or singleton clusters.

2.6 Priors and exposure selection

For the default fitting route, `Prior()` and Gaussian/probit defaults follow R `bkmr` 0.2.2. Gamma distributions below use shape and **rate**. The inverse-gamma density is proportional to $(\sigma^2)^{-a-1}\exp(-b/\sigma^2)$ for shape a and parameter b.

Quantity	Default
λ and λ_u, when clustered	Gamma(1, 0.1)
σ^2, continuous outcome	Inverse-gamma(0.001, 0.001)
Active r_m	$1/r_m \sim$ Uniform$(0, 100)$
Inclusion probability π	Beta(1,1)
$\boldsymbol{\beta}$	Flat prior

Count/survival `fit(..., prior=None)` instead uses $\lambda, \lambda_u \sim$ Gamma$(4, 4)$, independent $\boldsymbol{\beta} \sim N(\mathbf{0}, 25\mathbf{I})$, $\phi \sim$ Gamma$(2, 0.1)$ for NB2, and $\lambda_{0j} \sim$ Gamma$(0.1, 0.1)$ for survival. At fixed finite predictor values, these defaults give finite first and second prior-predictive moments of the count mean and relative hazard. A supplied `Prior` is used as specified. A partial dictionary overlays the selected family's defaults, for example `prior={"r_distribution": "gamma"}`. Warnings identify supplied amplitude or dispersion priors for which these moment guarantees do not hold. Assess prior sensitivity; large finite moments can still be numerically unstable. `sigma2` is only a stored unit-variance convention for new families and is omitted from their summaries.

The inverse-uniform relevance prior has an unbounded upper tail. A finite-chain average of r_m does not imply that its posterior expectation exists. Inspect quantiles and mixing; assess prior sensitivity for the application. `Prior(r_distribution="gamma")` and `"unif"` provide other positive priors. The gamma parameters use mean and standard deviation. Uniform bounds apply to r_m for `"unif"` and to $1/r_m$ for `"invunif"`.

With component selection, $\delta_m \sim$ Bernoulli(π) indicates whether $r_m > 0$. Excluded exposures have $r_m = 0$. Group selection uses the groups S_g and indicator ω_g from Bobb et al. (2015): each active group contains **one** selected exposure, chosen uniformly within that group. KernelMix does not offer unequal within-group prior probabilities. A group PIP is its posterior inclusion probability; a conditional PIP is an exposure's inclusion probability given that its group is active. A never-selected group has an undefined conditional PIP. PIPs depend on the model and priors; they do not measure causal importance.

2.7 Inputs and computation

`fit` adds an intercept and standardizes exposures by default. Standardization uses the retained training mean and sample standard deviation. Prediction reuses these values. Pass `intercept=False` or `standardize=False` to disable either. Do not include a constant adjustment column when adding an intercept. R `bkmr` does not automatically apply these Python preprocessing defaults.

`missing="raise"` rejects missing data. `missing="drop"` removes rows jointly across the outcome, exposures, covariates, cluster labels, and applicable offsets or event indicators. Retained original row indices are stored in `result.data["rows"]`. Complete-case fitting does not correct bias caused by missing data. Covariates must be numeric; categorical encoding is the caller's responsibility.

Chains run sequentially with separate random streams. `draws` counts saved draws per chain after `warmup`; `thin` controls the spacing of saved states. Proposals are fixed during sampling. `sampling_info` records acceptance counts and runtime. The default Gaussian sampler integrates out h during parameter sampling. The default binary sampler also samples correlated, sign-constrained latent outcomes; its predictions condition on those saved latent outcomes.

Default count/survival fits sample a noncentered GP with elliptical slice sampling, log-scale Metropolis updates and conjugate baseline updates. Eigenvalues below $100n\epsilon \max(1, \lambda_{\max}(\mathbf{K}))$, where ϵ is machine precision, define numerical null directions. No diagonal jitter is added. Prediction uses the same rank rule and the saved latent function. Saved auxiliary white coordinates may use different eigenvector signs on another platform; archive validation checks mathematical support rather than those signs.

Structured fits instead sample the GP and other normal-prior coordinates jointly by elliptical slice sampling, with log-scale Metropolis updates for amplitudes, relevance, dispersion and survival baseline rates. Gaussian residual variance has a conditional inverse-gamma update. Their priors and covariance options are defined in the structured guide.

Dense matrix factorization requires roughly cubic work in n per parameter update. Working-memory checks reject oversized fitting or prediction requests; they are protective estimates, not a promise about peak process memory.

2.8 Prediction and contrasts

`predict_h` draws h jointly over requested profiles. `predict(kind="mean")` returns outcome means, binary probabilities, or relative hazards for survival. `predict_survival` gives survival probabilities or hazards on a time grid. `predict(kind="observation")` also includes new observation variation for Gaussian, binary and response-scale count predictions. Survival observation sampling is not implemented. The latent binary scale is the underlying normal scale.

Contrasts subtract **joint** posterior draws so that covariance is retained. `overall_effect` compares all-exposure quantile profiles with a reference profile. `single_exposure_effects` changes one exposure while holding the others fixed. `interaction_effects` returns a difference of two single-exposure contrasts. Curves and surfaces vary one or two exposures; others are held fixed. Their intervals are equal-tailed, pointwise credible intervals, not simultaneous bands. On the response scale, adjustment covariates default to their retained training means unless `X` supplies a different profile.

Marginal quantile profiles may be uncommon in correlated data. Check exposure support before interpreting these contrasts. Causal interpretation requires additional identification assumptions; fitting a model does not supply them.

2.9 Diagnostics and scope

`diagnostics()` reports rank-normalized/folded split $\widehat{R}$, bulk effective sample size and tail effective sample size. Large $\widehat{R}$, small effective sample sizes, or constant sampled traces need investigation. Fixed parameters are excluded from these diagnostics. Trace plots and sensitivity checks remain necessary; favorable diagnostics do not prove convergence.

KernelMix supports Gaussian, probit, Poisson, NB2 and right-censored piecewise-exponential survival models with independent Gaussian random intercepts. The structured interface adds alternative kernels, finite GP bases, random slopes, Gaussian residual correlation and variance regression. It uses separately defined proper priors and a direct GP amplitude. Numerical comparisons cover selected fixed states and small analytic posterior targets. They do not establish general frequentist coverage or reproduce the original paper's studies.

3 An exposure-mixture analysis

KernelMix fits Bayesian kernel machine regression (BKMR) for continuous, binary, count and right-censored survival outcomes. The exposure-response function, called `h`, can include nonlinear associations and interactions. Adjustment covariates enter linearly.

This tutorial uses small, deterministic example data. Run its Python blocks in order after installing the source package as described in the README. The short chains make the examples quick to run. They are too short to support inference. For an analysis, choose longer runs, inspect mixing and effective sample sizes, and check sensitivity to prior choices.

3.1 Define the data

Each row is an observation. Columns of `Z` are exposures; columns of `X` are adjustment covariates. Keep `Z` in its original units. The fit stores the training means and sample standard deviations and reuses them for prediction.

```
import numpy as np
import kernelmix as km

t = np.linspace(-1.0, 1.0, 12)
Z = np.column_stack((1.5 + t, 2.0 + np.sin(3 * t), 10.0 + np.cos(5 * t)))
X = t[:, None]
y = 0.3 + np.sin(2 * t) + 0.2 * t**2
names = ["Exposure A", "Exposure B", "Exposure C"]
```

The default adds an intercept, so `X` should not include a constant column. Missing values raise an error. Use `missing="drop"` only when joint complete-case analysis is appropriate; that option drops a row if its outcome, exposure or adjustment value is missing.

3.2 Fit and inspect a continuous outcome

Component selection allows each exposure to enter or leave the model. Its posterior inclusion probability (PIP) is the proportion of saved states in which that exposure is included.

```
gaussian = km.fit(
    y, Z, X,
    family="gaussian", selection="component",
    exposure_names=names, covariate_names=["Adjustment"],
    draws=12, warmup=12, chains=2, seed=19,
)
print(gaussian.summary())
parameter_rows = gaussian.summary_table()
pip_rows = gaussian.inclusion_probabilities()
checks = gaussian.diagnostics()
```

The summary gives posterior means, standard deviations and pointwise credible intervals. Diagnostics include split R-hat, which compares chains, and effective sample sizes, which account for dependence between saved draws.

These checks help detect sampling problems; passing them does not establish that the model fits the scientific question. PIPs depend on the selection model and priors and are not causal importance scores.

3.3 Predict at exposure profiles

Supply new exposures in the training column order and their original units. New adjustment rows contain the raw `X` columns, without an intercept. They are required when adjustments were fitted.

```
Znew = Z[:3].copy()
Xnew = X[:3].copy()
mean_prediction = gaussian.predict(Znew, Xnew, seed=20)
observation_prediction = gaussian.predict(
    Znew, Xnew, kind="observation", seed=20,
)
h_prediction = gaussian.predict_h(Znew, seed=20)
prediction_rows = mean_prediction.to_records()
```

The default predicts outcome means. Observation predictions add residual noise and describe new outcomes. `predict_h` returns only the exposure function, without the intercept or adjustment contribution. Prediction objects retain joint draws, pointwise intervals and covariance between prediction rows. Calling `predict()` without new data uses the retained training rows and their adjustments.

3.4 Summarize exposure associations

Overall contrasts compare profiles at which every exposure is at a specified marginal quantile. Single-exposure contrasts change one exposure while fixing the others. These are conditional associations; a causal interpretation needs additional scientific assumptions.

```
overall = km.overall_effect(gaussian, reference=0.5, seed=21)
single = km.single_exposure_effects(gaussian, seed=21)
interaction = km.interaction_effects(
    gaussian, variables=["Exposure A", "Exposure B"], seed=21,
)
curve = km.univariate_response(
    gaussian, variable="Exposure A", points=7, seed=21,
)
surface = km.bivariate_response(
    gaussian, variables=("Exposure A", "Exposure B"), points=4, seed=21,
)
overall_rows = overall.to_records()
```

By default, these summaries describe changes in `h`. Curves and surfaces hold other exposures at their medians and subtract a common all-median reference profile. A centered value of zero means no change from that reference. `center=False` returns the uncentered curve or surface. Interval limits are pointwise posterior quantiles, not a simultaneous band across the whole curve.

Each contrast uses joint prediction draws for the profiles being compared. This retains their covariance. A profile compared with itself has exactly zero contrast. Marginal quantile combinations can be poorly represented among observed profiles, especially with correlated exposures; examine data support when interpreting them.

3.5 Fit a binary outcome

For a binary outcome, use `family="binomial"`. The probit link maps the latent mean to a probability with the standard normal cumulative distribution function. Outcomes must be zero or one.

```
y_binary = np.array([0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1])
binary = km.fit(
    y_binary, Z, X,
    family="binomial", exposure_names=names,
    covariate_names=["Adjustment"],
    draws=12, warmup=12, chains=2, seed=22,
)
probability_prediction = binary.predict(Znew, Xnew, seed=23)
probability_contrast = km.overall_effect(
    binary, scale="response", X=[0.0], seed=23,
)
```

Binary mean predictions are probabilities. With `kind="observation"`, predictions are zero-or-one outcomes. A response-scale contrast subtracts predicted probabilities after applying the probit transformation to each joint draw. It is not an odds ratio. In exposure summaries, `X` is one raw adjustment vector held constant at every profile. If it is omitted, response-scale summaries use the retained training covariate means. Latent-scale summaries use `h` alone and do not accept `X`.

3.6 Fit counts and include observation time

Use `"poisson"` for conditional Poisson counts or `"negative_binomial"` for additional conditional variation. An offset is the log observation time or another known exposure amount. It enters with coefficient one.

```
counts = np.array([0, 1, 0, 2, 1, 3, 1, 2, 4, 3, 5, 2])
observation_time = np.array([1, 1, 2, 1, 2, 1, 1, 3, 1, 2, 1, 1])
count_fit = km.fit(
    counts, Z, family="negative_binomial", offset=np.log(observation_time),
    draws=12, warmup=12, chains=2, seed=24,
)
count_mean = count_fit.predict(Z[:3], offset=np.log(observation_time[:3]), seed=25)
count_observation = count_fit.predict(
    Z[:3], offset=np.log(observation_time[:3]), kind="observation", seed=25,
)
```

`count_mean` describes expected counts for the supplied observation times. Observation predictions are nonnegative integer counts. Set `family="poisson"` to use the Poisson model. The negative-binomial `dispersion` parameter controls extra variation; it is separate from variation due to exposures or clusters. Response-scale exposure summaries use an offset of zero, so they compare counts per unit observation exposure. Their latent scale describes changes in `h`, which enter the log mean. These models use proper coefficient priors; see the model guide for their defaults.

3.7 Fit right-censored survival times

Each observation supplies a positive observed time and an event indicator: `1` means the event occurred; `0` means follow-up ended first. The model uses proportional hazards with a piecewise-constant baseline hazard.

```
observed_time = np.array([1.0, 1.4, 2.0, 2.6, 3.0, 3.8, 4.0, 4.7, 5.2, 6.0, 6.4, 7.0])
event = np.array([1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0])
survival_fit = km.fit(
    observed_time, Z, family="survival", event=event,
    baseline_breaks=[2.0, 4.0], intercept=False,
    draws=12, warmup=12, chains=2, seed=26,
)
survival = survival_fit.predict_survival(
    Z[:2], times=np.linspace(0, 7, 30), seed=27,
)
survival_ax = km.plot_survival(survival)
```

```
log_hr = km.overall_effect(survival_fit, quantiles=[0.75], reference=0.25, seed=27)
hr_draws = np.exp(log_hr.draws)
hr_interval = np.quantile(hr_draws, [0.025, 0.975], axis=(0, 1))
```

The baseline intervals are $(0, 2]$, $(2, 4]$ and $(4, \infty)$ in this example. An event exactly at a cut belongs to the interval on its left. The model assumes independent right censoring and proportional hazards. It does not support delayed entry, competing risks or time-varying covariates. Beyond observed follow-up, predictions extend the final constant baseline hazard. Such extrapolation needs scientific justification.

`predict_survival` returns survival probabilities, or hazards with `kind="hazard"`. Ordinary `predict` returns relative hazards for this family. Response-scale exposure contrasts therefore subtract relative hazards; they do not subtract survival probabilities. For hazard ratios, exponentiate each latent `h` contrast before summarizing it. The baseline and a fitted intercept share a scale, so this example omits the intercept.

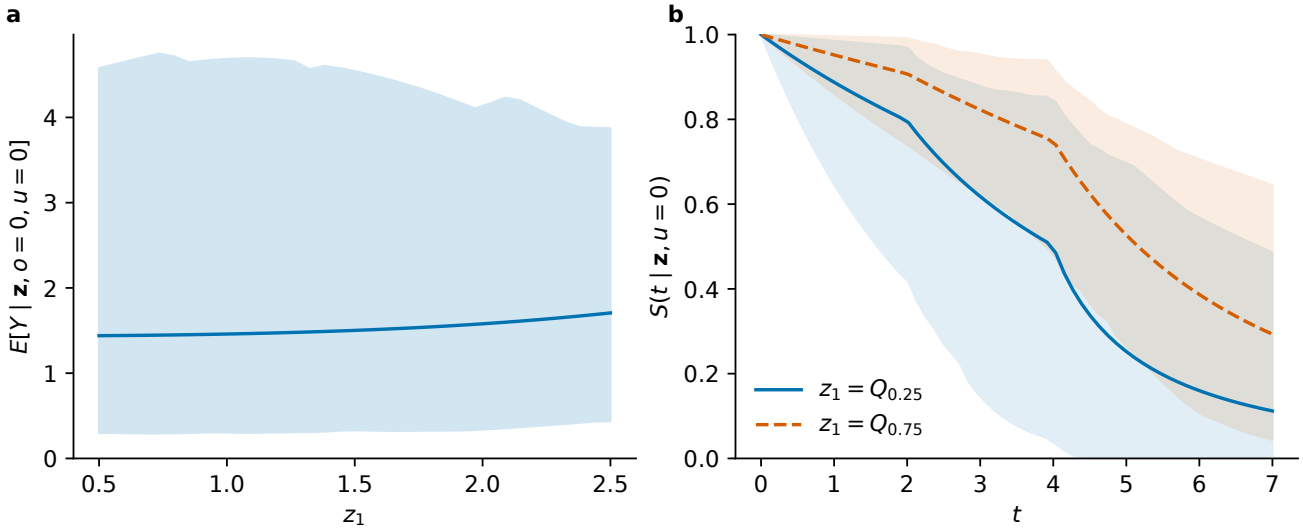


Figure 2: Count response and survival probabilities. (a) Negative-binomial mean counts as z_1 varies, with offset zero. (b) Survival probabilities at the first exposure's 25th and 75th percentiles. Other exposures are fixed at their medians and $u = 0$. Shading shows pointwise 95% credible intervals.

The figure uses the data above, 200 warmup and 200 saved draws per chain, two chains, and the same seeds. Predictions use 100 draws per chain. These short fits have mixing warnings and illustrate the outputs; they do not support inference or establish interval calibration. Figure values.

3.8 Account for clustered observations

`cluster` labels observations that share a Gaussian random intercept. It is separate from `groups`, which groups exposures for selection. All five families accept `cluster`.

```
cluster = ["A"] * 3 + ["B"] * 3 + ["C"] * 3 + ["D"] * 3
clustered_fit = km.fit(
    counts, Z, family="poisson", cluster=cluster,
    draws=12, warmup=12, chains=2, seed=28,
)
intercepts = clustered_fit.random_intercepts(seed=29)
cluster_prediction = clustered_fit.predict(
    Z[:3], cluster_new=["A", "new", "new"], seed=29,
)
```

Known prediction labels use their posterior random intercept. Repeated unseen labels share one new intercept per posterior draw. Omitting `cluster_new` fixes the prediction random intercept to zero; this is not an average over clusters. The exposure summaries also use a zero random intercept. This model captures shared intercept variation, not arbitrary within-cluster time dependence.

3.9 Plot the results

Install the optional Matplotlib dependency to use these functions. Each returns an axes object. The caller controls display and export.

```
trace_ax = km.plot_trace(gaussian)
pip_ax = km.plot_pips(gaussian)
curve_ax = km.plot_response(curve)
overall_ax = km.plot_overall(overall)
surface_ax = km.plot_surface(surface)

# Optional: display the figures in an interactive session.
# import matplotlib.pyplot as plt
# plt.show()
```

Curve and overall plots include pointwise posterior intervals. The surface plot displays the posterior mean only; its result object also retains lower and upper limits. Plot exposures separately when their units differ. The trace excludes warmup draws.

3.10 Save and load a fit

A fit includes its retained training data as well as saved states. Store and share it with the same care as the input data. The archive uses a non-pickle format.

```
from pathlib import Path
from tempfile import TemporaryDirectory

with TemporaryDirectory() as directory:
    path = Path(directory) / "mixture-fit.npz"
    gaussian.save(path)
    restored = km.BKMRFit.load(path)
    restored_prediction = restored.predict(Znew, Xnew, seed=20)
```

See the API reference for argument defaults and result fields. The method is described by Bobb et al. (2015); Bobb et al. (2018) describes exposure summaries and the probit extension. KernelMix's pairwise interaction summary and median-reference curve centering are specified in the API reference.

4 Structured models and GP approximations

Use these options when the analysis needs a different exposure kernel, a finite GP basis, random slopes, Gaussian residual structure, or extended survival records. `km.fit` returns a `StructuredFit` when one of these options is selected. The ordinary defaults retain the original fitting route. The underlying exposure model follows Bobb and colleagues, with the software extension; the additional structures below have their own stated definitions.

The examples below share eight fixed observations. Run the blocks in order. Four saved draws and four warmup iterations per chain show the workflow only; they cannot support inference. Longer runs, sampling diagnostics, prior sensitivity and checks of data support are needed for an analysis.

```
import numpy as np
import kernelmix as km

x = np.linspace(-1.0, 1.0, 8)
visit = np.tile(np.arange(4), 2)
cluster = np.array(["a"] * 4 + ["b"] * 4)
Z = np.column_stack((2 + x, 5 + np.sin(np.arange(8))))
X = np.cos(np.arange(8))[:, None]
y = 0.3 + x + 0.2 * np.sin(2 * np.arange(8))
slope = x[:, None]
V = (x - x.mean())[:, None]
demo = dict(draws=4, warmup=4, chains=2, seed=47)
```

All exposure inputs and explicit knots use original exposure units. Training standardization is stored and reused. Adjustment and random-slope inputs exclude the automatically added intercept.

4.1 Choose an exposure kernel

For exposure profiles $\mathbf{z}$ and $\mathbf{z}'$, define

$$q(\mathbf{z}, \mathbf{z}') = \sum_{m=1}^{M} r_m (z_m - z'_m)^2, \qquad r_m \geq 0.$$

The profiles use the fitted standardization when enabled. Setting $r_m = 0$ removes exposure m from the kernel.

`kernel`	Kernel at weighted squared distance q
`"gaussian"`	$\exp(-q)$
`"matern32"`	$(1 + \sqrt{6q})\exp(-\sqrt{6q})$
`"rational_quadratic"`	$(1 + q/\alpha)^{-\alpha}$

For the rational-quadratic kernel, α (`kernel_alpha`) is fixed, positive and defaults to one. A nondefault value requires `kernel="rational_quadratic"`. Increasing it makes the rational-quadratic kernel approach the Gaussian kernel. These are the Matérn and rational-quadratic definitions in GPML, Chapter 4, expressed using KernelMix's relevance weights.

```
matern = km.fit(y, Z, X, kernel="matern32", **demo)
rq = km.fit(y, Z, X, kernel="rational_quadratic", kernel_alpha=1.5, **demo)
curve = km.univariate_response(matern, variable=0, points=5, seed=48)
```

Structured fits store `lambda` as the **direct GP covariance amplitude**, corresponding to Bobb's τ: $\mathbf{h} \sim N(\mathbf{0}, \tau\mathbf{K})$. This also applies to structured Gaussian fits. The legacy Gaussian route instead stores the ratio τ/σ^2. Selecting a structured model therefore changes its prior specification as well as its covariance options. Exposure contrasts describe conditional associations; causal interpretation requires additional assumptions.

Structured adjustment coefficients, including an intercept when present, have independent normal priors with standard deviation `prior.beta_sd`, default five. Default GP amplitudes have Gamma shape four and rate four. Structured Gaussian residual variance defaults to inverse-gamma shape two and rate one, meaning its reciprocal has that Gamma distribution. A `prior` dictionary overrides only supplied fields on these structured defaults; a `Prior` object supplies all its fields directly. Inspect `fit.settings["prior"]` when comparing models. The legacy `cluster_lambda_mean` and `cluster_lambda_sd` fields do not govern structured random coefficients; their covariance prior is defined below.

4.2 Reduce GP matrix size

`approximation="dense"` uses the full kernel. `"pp"` uses a predictive process at a fixed set of knots. `"rff"` uses frozen random Fourier features.

```
pp = km.fit(y, Z, X, kernel="matern32", approximation="pp",
            n_components=4, **demo)
rff = km.fit(y, Z, X, kernel="matern32", approximation="rff",
             n_components=8, **demo)
pp_prediction = pp.predict(Z[:2], X[:2], seed=48)
```

For PP, `n_components` is the maximum number of automatic, deterministic max-min knots. Alternatively, supply `knots=Z[[0, 3, 5, 7]]`. Its covariance is $\tau\mathbf{K}_{ZU}\mathbf{K}_{UU}^{+}\mathbf{K}_{UZ}$, where U denotes the knots and $+$ is the numerical pseudoinverse. It omits the parent GP's conditional residual variance. Uncertainty can therefore be much smaller away from knots; this is the projected model described by Banerjee and colleagues, not an exact dense-GP uncertainty calculation.

For RFF, `n_components` counts frequency pairs, giving twice as many features. Paired sine/cosine features follow Rahimi and Recht. Frequencies are Gaussian for the Gaussian kernel, multivariate Student-t with three degrees

of freedom for Matérn, and a Gaussian/Gamma scale mixture for RQ. Each mixture scale is shared across all coordinates of one frequency vector. Base draws stay fixed throughout sampling and are saved with the fit. Inference is conditional on that realized feature map; a fixed feature count does not establish approximation accuracy.

During fitting, PP/RFF use observation-by-feature and feature-by-feature matrices. Joint predictions and exposure contrasts still return dense matrices over the requested prediction rows; keep these requests small or use suitable batches. Large correlated Gaussian residual blocks can still require dense calculations. Dense/PP eigensystems treat eigenvalues at most `100 * dimension * machine_epsilon * largest_eigenvalue` as numerical null modes. Saved dense training values and PP knot values preserve prediction coordinates across reloads.

4.3 Model random intercepts and slopes

`cluster` identifies observation groups; exposure-selection `groups` has a separate meaning. `random_slopes` adds columns after the random intercept. All five outcome families support these coefficients.

```
slopes = km.fit(y, Z, X, cluster=cluster, random_slopes=slope, **demo)
coefficients = slopes.random_effects(["a", "new"], seed=49)
intercepts = slopes.random_intercepts(["a", "new"], seed=49)
prediction = slopes.predict(
    Z[:2], X[:2], cluster_new=["a", "new"], random_slopes_new=slope[:2], seed=49,
)
fixed_slopes = km.fit(
    y, Z, X, cluster=cluster, random_slopes=slope,
    random_covariance=[[0.5, 0.1], [0.1, 0.3]], **demo,
)
```

`random_effects()` returns joint coefficient draws; `predict` combines them with the new slope design. A fixed covariance must be positive definite and match `[intercept, slopes]`. Otherwise, the full covariance is estimated. Each standard deviation is `random_scale * Phi(raw)`, where `raw` is standard normal and `Phi` is the standard normal cumulative distribution function. Thus SDs are uniform between zero and `random_scale`, default two. A normalized lower-triangular factor with normal off-diagonal coordinates supplies a proper correlation prior. It is neither an LKJ prior nor a uniform prior on correlation matrices.

Known labels use saved coefficients. Each distinct new label receives one fresh vector per draw, shared across its rows. Omitting `cluster_new` sets random effects to zero; on nonlinear links this differs from averaging over new clusters. Supply `random_slopes_new` when predicting their contribution at new rows. Redundant random-design columns are rejected, but few clusters or little slope variation can still leave covariance estimates strongly dependent on priors.

Count/probit inputs retain their usual meanings. For example, an NB2 model can use the same slope workflow:

```
counts = np.array([0, 1, 2, 1, 0, 2, 1, 3])
nb = km.fit(counts, Z, X, family="negative_binomial", cluster=cluster,
            random_slopes=slope, approximation="rff", n_components=8, **demo)
count_means = nb.predict(Z[:2], X[:2], seed=50)
```

Count `offset` remains a log observation-exposure offset. Response predictions are count means or, with `kind="observation"`, sampled counts. Response-scale exposure contrasts use zero offset and zero random effects. Survival contrasts describe differences in relative hazards; they are not hazard ratios or survival-probability differences.

4.4 Separate Gaussian variance and correlation

With `variance_design=V`, $\mathrm{Var}(\epsilon_i \mid \sigma^2, \boldsymbol{\gamma}) = \sigma^2 \exp(\mathbf{v}_i^\top \boldsymbol{\gamma})$, where $\mathbf{v}_i$ is row i of V and $\boldsymbol{\gamma} \sim N(\mathbf{0}, \mathbf{I})$. No intercept is added to `V`. Its columns must be independent and their span must exclude a constant, because `sigma2` already controls the baseline variance. This is an exponential variance model with the package's own parameterization; compare nlme's variance-function definition.

```
ar = km.fit(
    y, Z, X, cluster=cluster, variance_design=V,
    correlation="ar1", observation_time=visit, correlation_parameter=0.4, **demo,
)
future_rows = np.array([3, 7])
future_mean = ar.predict(
    Z[future_rows], X[future_rows], cluster_new=["a", "b"],
    observation_time_new=[4, 4], variance_design_new=V[future_rows], seed=51,
)
residual_variance = ar.predict_residual_variance(
    Z[future_rows], variance_design_new=V[future_rows],
)
exponential = km.fit(
    y, Z, X, cluster=cluster, correlation="exponential",
    observation_time=visit / 2, **demo,
)
```

Residual correlation applies only to Gaussian outcomes and requires clusters. AR1 times are distinct integers within each cluster; exponential times are distinct finite values. AR1 correlation at a time gap is `rho**gap`, with `-1 < rho < 1`; exponential correlation is `exp(-gap/range)`, with positive range. See nlme's AR1 definition and exponential definition. A supplied `correlation_parameter` is fixed. Otherwise, AR1 uses `rho=tanh(raw)` and exponential uses `range=exp(raw)`, each with a standard-normal prior on `raw`.

For a known cluster, correlated prediction conditions on training residuals after subtracting its fitted structural predictor. Mean prediction includes the conditional residual mean; observation prediction also includes conditional residual uncertainty. An unseen cluster has no training residual cross-correlation. To condition predictions for training rows on the recorded process, supply their original `cluster_new` labels; training times and slope rows are reused. Omitting labels supplies no training residual cross-correlation. `predict_h` and exposure contrasts retain the exposure/structural target rather than this residual-process target.

A fixed correlation matrix must cover all original rows, have unit diagonal, be positive definite, and be zero between clusters. Supply new-by-training and new-by-new correlations to condition new residuals:

```
R = np.where(cluster[:, None] == cluster[None, :],
             0.3 ** np.abs(visit[:, None] - visit[None, :]), 0.0)
fixed = km.fit(y, Z, X, cluster=cluster, correlation=R, **demo)
cross = np.array([
    np.where(cluster == label, 0.3 ** (4 - visit), 0.0)
    for label in ("a", "b")
])
fixed_prediction = fixed.predict(
    Z[future_rows], X[future_rows], cluster_new=["a", "b"],
    correlation_cross=cross, correlation_new=np.eye(2), seed=52,
)
```

With fixed correlation and no custom cross matrix, new residuals are independent of training residuals. The combined training/new correlation must be positive semidefinite. Residual variance predictions exclude uncertainty in the GP and random coefficients. Random effects and long-range residual correlation can be hard to distinguish with sparse repeated measurements.

4.5 Encode delayed entry and competing events

For survival, `y` is an exact/censoring time or an interval lower bound. `event=0` means right censoring; positive integers `1..n_causes` identify causes. The first terminal event is modeled; recurrent events and general state transitions are excluded. Baseline intervals are right closed, so an event exactly at a cut belongs to its left interval.

```
time = np.array([0.8, 1.4, 2.2, 3.2, 1.1, 1.9, 2.8, 3.6])
event = np.array([1, 0, 2, 1, 0, 2, 0, 1])
entry = np.array([0.0, 0.2, 0.5, 0.4, 0.1, 0.3, 0.6, 0.2])
survival = km.fit(
    time, Z, X, family="survival", event=event, entry=entry, n_causes=2,
    baseline_breaks=[1.5, 2.5], intercept=False, **demo,
)
times = np.array([0.0, 1.0, 2.0, 3.0, 4.0])
s = survival.predict_survival(Z[:2], X[:2], times=times, entry=0.2, seed=53)
cif = survival.predict_cumulative_incidence(Z[:2], X[:2], times=times, entry=0.2, seed=53)
cause_two = survival.predict_cumulative_incidence(
    Z[:2], X[:2], times=times, cause=2, entry=0.2, seed=53,
)
hazard_two = survival.for_cause(2).predict_survival(
    Z[:2], X[:2], times=times, kind="hazard", entry=0.2, seed=53,
)
cause_contrast = km.overall_effect(survival.for_cause(2), seed=53)
```

For cause k, the hazard is $\lambda_k(t \mid \mathbf{z}_i, \mathbf{x}_i) = \lambda_{0k}(t) \exp\{h_k(\mathbf{z}_i) + \mathbf{x}_i^\mathsf{T} \boldsymbol{\beta}_k\}$, with random-coefficient terms added when supplied. The cause-specific exposure functions, adjustment coefficients, amplitudes and baselines are independent conditional on the shared exposure relevance and selection parameters; they need not be independent after those parameters are integrated out.

Survival is the probability of no event of any cause. A cumulative incidence function (CIF) is the probability of a particular cause occurring by a time, accounting for all competing causes. Joint draws satisfy survival plus the sum of CIFs equals one. This distinction is explained in the R survival competing-risk guide. `cif` retains all causes; `cif.for_cause(2)` selects one. A fit's `for_cause(2)` view instead selects its exposure function, relative-hazard predictions and contrasts. `km.plot_survival(cause_two)` plots a selected CIF with pointwise intervals when Matplotlib is installed.

Prediction `entry` conditions on survival to that time; before it, survival is one and CIFs are zero. Static-profile prediction extends the final baseline hazard beyond follow-up. Baseline cuts are held fixed; their selection uncertainty is excluded. A survival intercept and baseline share a scale, so `intercept=False` is useful unless their separate prior-dependent roles are intended.

These delayed-entry fits describe the **included entrant cohort**, conditional on its recorded entry and predictor design. GP/random-effect priors do not estimate an inception-population frailty distribution or correct its change under survival-to-entry selection. New-cluster predictions inherit this cohort-conditional model.

4.6 Record left or interval censoring

Supply `upper > y` for a finite event interval. A left-censored record has lower bound zero and entry zero. An exact event uses `upper == y` or omits `upper`; right censoring uses `event=0` and may use infinite upper bound. Unknown cause `event=-1` is allowed only for a finite interval. Set `n_causes` explicitly when all observed interval causes are unknown. These inputs use explicit KernelMix codes rather than the alternative status codes of R's Surv.

```
lower = np.array([0.0, 1.4, 1.8, 3.2, 1.1, 1.5, 2.8, 3.0])
upper = np.array([0.8, np.inf, 2.2, 3.2, np.inf, 1.9, np.inf, 3.6])
interval_event = np.array([1, 0, 2, 1, 0, -1, 0, 1])
interval = km.fit(
    lower, Z, X, family="survival", event=interval_event, upper=upper,
    entry=np.zeros(8), subject=np.arange(8), n_causes=2,
    baseline_breaks=[1.5, 2.5], intercept=False, **demo,
)
```

The likelihood assumes noninformative censoring and inspections, and ignorable cause recording conditional on

modeled information. Unknown-cause integration does not model a cause-dependent recording mechanism. An interval row assumes its supplied predictors apply throughout the uncertain event interval.

4.7 Supply known covariate paths

Repeated `subject` labels identify successive `(entry, y]` risk intervals. Rows must be chronological, contiguous and nonoverlapping within a subject; only its last row may have an event or a finite censoring interval. Later row starts split an existing history rather than representing new cohort entry. Known subject histories with missing required values are dropped as a whole under `missing="drop"`; this preserves recorded histories but does not remove missing-data bias.

```
subject = np.repeat(np.arange(4), 2)
start = np.tile([0.0, 1.0], 4)
stop = np.tile([1.0, 3.0], 4)
path_event = np.array([0, 1, 0, 0, 0, 2, 0, 1])
path_fit = km.fit(
    stop, Z, X, family="survival", event=path_event, entry=start,
    subject=subject, n_causes=2, baseline_breaks=[1.5, 2.5],
    intercept=False, **demo,
)
path_inputs = dict(start=start, stop=stop, subject_new=subject,
                   times=[0.0, 1.0, 2.0, 3.0], seed=54)
path_survival = path_fit.predict_survival_path(Z, X, **path_inputs)
path_cif = path_fit.predict_survival_path(
    Z, X, kind="cumulative_incidence", **path_inputs,
)
path_cause_two = path_fit.predict_survival_path(
    Z, X, kind="cumulative_incidence", cause=2, **path_inputs,
)
```

Path prediction integrates only the supplied piecewise-constant predictors. Paths must cover requested times through each subject's last stop; the last row is never extended automatically. Use `kind="hazard"` for evaluated cause-specific hazards. Returned rows follow the first occurrence of `subject_new`. Repeated new cluster labels share their random vector across path intervals when supplied.

Covariates must be known before the risk interval they describe. Measurements observed at an event or later inspection cannot be copied backward to create a past path. These functions neither interpolate missing trajectories nor forecast future covariates. Intervals are evaluated jointly across times and causes; the reported credible intervals are pointwise, not simultaneous bands.

4.8 Set starting values

Structured `initial` dictionaries accept `beta`, `gp_weight`, `r`, `delta`, `pi`, `lambda` and `sigma2`; NB2 adds `dispersion` and survival adds `baseline_hazard`. Omitted values receive defaults. With K causes (one for nonsurvival models), p adjustment coefficients including the intercept, and b retained basis coordinates:

Key	Required shape and restriction
`beta`	(K, p); a length-p vector is also accepted when $K = 1$
`gp_weight`	(K, b); a length-b vector is also accepted when $K = 1$
`r`, `delta`	One value per exposure; relevance and zero/one inclusion must agree
`lambda`	Positive scalar or length-K vector of direct amplitudes
`sigma2`	Positive scalar; fixed at one outside Gaussian models
`pi`	Inclusion probability; one without selection
`dispersion`	Positive scalar for NB2
`baseline_hazard`	Positive (K, J) array for J baseline intervals

Random coefficients, covariance coordinates, variance coefficients and inferred residual-correlation coordinates start at their internal defaults. Their raw sampling coordinates are not an initialization interface. Dense basis dimension

is the retained observation count, PP dimension is the knot count, and RFF dimension is twice `n_components`; numerical null columns are retained. Omit `gp_weight` to use zero starting coordinates.

Saved structured arrays have leading `(chain, draw)` axes. They include cause-indexed `beta`, `lambda` and `gp_weight`, shared `r`, `delta` and `pi`, and Gaussian `sigma2`. Dense/PP fits retain coordinate-independent `gp_anchor` values; RFF fits retain their frozen feature map. Applicable models also save `random_coeff`, `random_covariance`, `gamma`, `correlation_parameter`, `dispersion` and `baseline_hazard`. Use result methods for prediction rather than interpreting internal coordinates as exposure effects.

5 API reference

Import the public API with `import kernelmix as km`. Arrays are NumPy arrays; pandas is not required. Examples that connect these functions are in the tutorial.

5.1 Fitting

```
km.fit(
    y, Z, X=None, *, family="gaussian", draws=1000, warmup=1000,
    chains=4, thin=1, seed=None, standardize=True, intercept=True,
    selection="none", groups=None, prior=None, r_fixed=None, missing="raise",
    exposure_names=None, covariate_names=None, initial=None, proposal_scales=None,
    cluster=None, offset=None, event=None, baseline_breaks=None,
    kernel="gaussian", kernel_alpha=1, approximation="dense", n_components=32,
    knots=None, random_slopes=None, random_covariance=None, random_scale=2,
    correlation=None, observation_time=None, correlation_parameter=None,
    variance_design=None, entry=None, upper=None, subject=None, n_causes=None,
)
```

Returns `BKMRFit`, or its `StructuredFit` subclass when a structured option is selected. Structured models defines the additional inputs, priors, shapes and predictions. Original default fitting paths remain unchanged.

Family	Required outcome inputs	Mean or probability
Gaussian	Numeric `y`	Continuous outcome mean
Binomial	`y` equal to 0 or 1	Probit probability
Poisson / negative binomial	Nonnegative integer `y`; `offset` when needed	Count mean, including the log offset
Survival	Default: positive observed times and binary `event`; structured: cause codes, entry and censoring bounds	Relative hazard from `predict`; survival/hazard from `predict_survival`

`cluster` supplies one string or integer label per observation. Missing labels participate in `missing="drop"`. Integer `1` and string `"1"` are distinct. `offset` is a finite vector of log observation exposures for count families. `event` is required only for survival; `baseline_breaks` supplies internal positive, strictly increasing cuts. Omit cuts to use event-time deciles, or pass `[]` for a constant baseline hazard. Family-specific arguments are rejected elsewhere.

Input	Meaning
`y`, `Z`, `X`	Outcome vector, exposure matrix and optional adjustment matrix, with matching rows. Supply raw adjustment columns without an intercept when `intercept=True`.
`family`	`"gaussian"`, `"binomial"` (probit), `"poisson"`, `"negative_binomial"` (NB2) or `"survival"` (piecewise-exponential proportional hazards).

Input	Meaning
`draws`, `warmup`, `chains`, `thin`	Retained states per chain, discarded iterations, sequential chains and spacing between retained states. Total iterations per chain are `warmup + draws * thin`. At least two retained states are required.
`seed`	Nonnegative integer or `None`; uses local random generators without changing NumPy's global random state.
`standardize`	Center exposures and divide by retained-training sample standard deviations. Constant exposures are invalid when this is enabled.
`selection`	`"none"`, `"component"` or `"group"`. Group selection requires one `groups` label per exposure and permits at most one active exposure in each included group.
`prior`	`Prior`, a dictionary of its fields, or `None` for defaults.
`r_fixed`	Finite nonnegative vector fixing exposure relevance values; disallows variable selection.
`missing`	`"raise"` or joint complete-case `"drop"`. Infinity remains invalid except for right-censoring upper bounds.
Names	Optional unique names for exposure and raw adjustment columns. Otherwise use input DataFrame columns, or generated names.
`initial`	Shared chain initialization dictionary. Basic-sampler keys: `beta`, `r`, `lambda`, `delta`, `pi`. Gaussian/probit use `sigma2` and binary `ystar`; new families use `latent_white`, with NB2 `dispersion` or survival `baseline_hazard`. Clusters add `lambda_cluster` and new-family `u_white`. Omitted values receive defaults; inapplicable keys raise errors. Structured models use cause-indexed coefficients and `gp_weight`; see their guide.
`proposal_scales`	Positive log-scale standard deviations for `r`, `lambda`, and applicable `cluster_lambda` or NB2 `dispersion`. Defaults are `0.3`; scales stay fixed.

The adjustment design must have full column rank and fewer columns than retained observations. The default binary sampler rejects complete or quasi-complete separation under its flat coefficient prior; structured fits use a proper normal prior. Default fitting uses dense covariance matrices, with quadratic memory and cubic linear-algebra cost. Predictive-process and random-Fourier options replace the GP with the selected finite basis; dense residual correlation blocks can remain costly. Successful fitting does not establish convergence.

5.1.1 Prior configuration

```
km.Prior(
    r_distribution="invunif", r_mean=5.0, r_sd=5.0,
    r_lower=0.0, r_upper=100.0, lambda_mean=10.0, lambda_sd=10.0,
    sigma2_shape=0.001, sigma2_rate=0.001,
    inclusion_alpha=1.0, inclusion_beta=1.0,
    cluster_lambda_mean=10.0, cluster_lambda_sd=10.0, beta_sd=5.0,
    dispersion_shape=2.0, dispersion_rate=0.1,
    baseline_shape=0.1, baseline_rate=0.1,
)
```

`r_distribution` accepts `"gamma"`, `"unif"` and `"invunif"`. The inverse-uniform option places the uniform distribution on `1/r`, so its bounds apply to the reciprocal of an active relevance value. Excluded exposures have

relevance zero. Gamma priors use mean and standard deviation; the residual variance uses inverse-gamma shape and rate. Inclusion probabilities use a beta prior. The residual variance prior is used for Gaussian outcomes.

Count/survival `prior=None` uses `lambda_mean=1`, `lambda_sd=.5`, `cluster_lambda_mean=1`, `cluster_lambda_sd=.5`; the other fields retain their listed defaults. A dictionary overlays family defaults. A `Prior` object supplies all its values explicitly, including its listed amplitude defaults. `beta_sd` controls the proper coefficient prior for count/survival. Dispersion and baseline parameters use gamma shape/rate; other-family fields are unused as documented.

5.2 Structured model results

Task	Method or helper
Exact alternative kernels	`kernel_matrix(Z1, Z2=None, r=None, kernel="gaussian", alpha=1)`
Intercept/slope coefficient draws	`fit.random_effects(cluster_new=None, ...)`
Gaussian residual variances	`fit.predict_residual_variance(Znew=None, variance_design_new=None, ...)`
Cause-specific h and effect summaries	`fit.for_cause(cause)`
Competing-risk probabilities	`fit.predict_cumulative_incidence(..., times=..., cause=None)`
Time-varying predictor paths	`fit.predict_survival_path(..., start=..., stop=..., subject_new=..., times=...)`

`StructuredFit` retains the shared summary, sample, diagnostic, contrast and save/load methods. `RandomCoefficientPrediction` stores coefficient draws; `CompetingRiskPrediction.for_cause(k)` returns a curve accepted by `plot_survival`. Additional inputs, initialization shapes and runnable examples are in the structured guide.

5.3 Fit results and diagnostics

`BKMRFit` retains `data`, `samples`, `settings` and `sampling_info`. Properties `family`, `exposure_names` and `covariate_names` describe the fitted inputs. Stored arrays are copied and read-only; extracted samples are independent copies.

Method	Return and use
`fit.summary(probability=0.95)`	Text summary of parameter means, SDs, equal-tailed intervals, available PIPs and diagnostic messages.
`fit.summary_table(probability=0.95)`	List of dictionaries with `parameter`, `mean`, `sd`, `lower`, `upper`.
`fit.posterior_samples(parameter=None, combine_chains=False)`	Dictionary of arrays, or one named parameter array. Leading axes are chain and retained draw; combining chains merges those axes.
`fit.inclusion_probabilities()`	Exposure names and PIPs for selection fits. Group fits also return group PIP and within-group conditional PIP; the conditional value is undefined (`NaN`) if a group was never selected. Raises without selection.
`fit.diagnostics()`	Parameter diagnostics and messages. Uses rank-normalized/folded split R-hat and bulk/tail effective sample sizes.
`fit.save(path)`	Save training data, settings and states in a compressed non-pickle archive. Replaces the destination atomically.
`km.BKMRFit.load(path, *, max_bytes=512 * 1024 * 1024)`	Load and validate an archive, limiting total uncompressed member size.

Method	Return and use
`km.diagnose(values)`	Diagnostics for one finite scalar-parameter array shaped `(chains, draws)`.

Default-route saved parameters include `beta`, `r`, `lambda`, `sigma2`, `delta` and `pi`; its binary fits also retain `ystar`. Structured arrays are described in the structured guide. `delta` records exposure inclusion. `beta` includes the added intercept when fitted. No-selection fits store `pi=1`. Diagnostics cannot establish model validity; R-hat requires multiple chains and short or constant sequences can yield undefined diagnostics.

5.4 Prediction

```
fit.predict_h(Znew=None, *, probability=0.95, max_draws=None, seed=None)
fit.predict(
    Znew=None, Xnew=None, *, kind="mean", scale="response",
    probability=0.95, max_draws=None, seed=None, offset=None, cluster_new=None,
)
fit.posterior_moments_h(Znew=None, max_draws=None)
```

All new exposure values use original units and the training column order. Named input columns must match the stored names and order. Training standardization is reused. `Xnew` contains raw adjustment columns without the intercept; it is required for new profiles when adjustments were fitted. With no new data, retained training rows and their adjustments are used.

Call	Quantity
`predict_h`	Joint draws of `h`, excluding linear adjustment and random intercepts.
`predict(..., kind="mean", scale="response")`	Gaussian/count means, binary probabilities, or survival relative hazards.
`predict(..., kind="observation", scale="response")`	Gaussian, binary or count future observations; survival is unsupported.
`predict(..., kind="mean", scale="latent")`	`h` plus linear adjustment and requested random intercept; count log offsets are included. This is before the probit/log-link transformation.
`predict(..., kind="observation", scale="latent")`	Gaussian outcomes or binary latent outcomes including residual noise; binary residual variance is one. Count/survival latent observation draws are unsupported.
`posterior_moments_h`	Dictionary with deterministic integrated `mean` and `covariance` for `h`, averaging conditional moments over saved states.

`max_draws` selects at most that many evenly spaced saved states **per chain**, with a minimum of two. It does not refit. `seed` controls conditional prediction draws. A `Prediction` contains `draws` shaped `(chains, selected_draws, rows)`, `mean`, `sd`, `lower`, `upper`, joint `covariance`, `probability`, `scale` and `kind`. Its `to_records()` method returns one summary dictionary per row. Intervals are equal-tailed pointwise posterior intervals; observation intervals also include observation noise.

5.5 Cluster and survival results

```
fit.random_intercepts(cluster_new=None, *, probability=.95, max_draws=None, seed=None)
fit.predict_survival(
    Znew=None, Xnew=None, *, times, kind="survival", cluster_new=None,
    probability=.95, max_draws=None, seed=None,
)
km.plot_survival(prediction, row=0, ax=None)
```

`random_intercepts` returns a `Prediction` in training-cluster order by default, or supplied label order. Known and unseen labels follow the policy below. `predict_survival` requires finite, nonnegative, strictly increasing `times`. Its `SurvivalPrediction.draws` has axes `(chain, draw, row, time)`; `mean`, `sd`, `lower` and `upper` have shape `(row, time)`. Fields `times`, `probability` and `kind` identify the grid and summary. `to_records()` returns one row per profile/time pair. `kind="hazard"` evaluates hazards on that grid. The plot shows a survival curve, or hazard points and intervals without inventing step locations between a sparse set of times.

`predict` additionally accepts `offset=None` and `cluster_new=None`. For training count prediction, omitted offsets reuse retained offsets. For new rows, offsets are required if they were supplied during fitting. Response-scale count observations are Poisson or NB2 draws. Latent count observations and survival observation draws are unsupported and raise an error.

Omitted `cluster_new` fixes the intercept to zero, including for training predictions. Known labels use their posterior; unseen labels use a new Gaussian intercept. Equal labels share a draw. `predict_h` always excludes random intercepts. Default Gaussian/probit prediction samples `h + u` jointly when labels are supplied, preserving posterior covariance.

For the default count/survival route, samples include `h` and auxiliary `latent_white`; negative-binomial fits also include `dispersion`; survival fits include `baseline_hazard`. Default-route clustered fits include `lambda_cluster`, and count/survival fits also retain `u`. Default Gaussian/probit random intercepts are integrated out while fitting and drawn conditionally by `random_intercepts`. `sigma2=1` is only a shared latent convention for count/survival and is omitted from their summaries. Saved white coordinates refer to the sampler's eigensystem; use saved `h` for portable prediction.

5.6 Exposure contrasts and curves

```
km.overall_effect(
    fit, quantiles=(0.25, 0.5, 0.75), reference=0.5, scale="latent",
    X=None, probability=0.95, max_draws=None, seed=None,
)

km.univariate_response(
    fit, variable=None, grid=None, points=50, fixed_quantile=0.5,
    center=True, scale="latent", X=None, probability=0.95,
    max_draws=None, seed=None,
)

km.bivariate_response(
    fit, variables=(0, 1), grids=None, points=20, fixed_quantile=0.5,
    center=True, scale="latent", X=None, probability=0.95,
    max_draws=None, seed=None,
)

km.single_exposure_effects(
    fit, quantiles=(0.25, 0.75), fixed_quantiles=(0.25, 0.5, 0.75),
    scale="latent", X=None, probability=0.95, max_draws=None, seed=None,
)
```

```
km.interaction_effects(
    fit, variables=None, quantiles=(0.25, 0.75), fixed_quantile=0.5,
    scale="latent", X=None, probability=0.95, max_draws=None, seed=None,
)
```

Function	Quantity and result
`overall_effect`	Each all-exposure quantile profile minus the all-exposure `reference` profile. Returns `EffectResult`.
`univariate_response`	One named or zero-based indexed exposure, or all exposures with `variable=None`. Other exposures stay at `fixed_quantile`. Returns one joint `CurveResult`.
`bivariate_response`	A Cartesian grid for two distinct exposures. Other exposures stay at `fixed_quantile`. Returns `SurfaceResult`.
`single_exposure_effects`	High-minus-low quantile contrast for each exposure at every background `fixed_quantiles` value. Returns `EffectResult`.
`interaction_effects`	All unordered pairs from the selected names/indices, or all exposures by default. Returns `EffectResult`. For each pair the contrast is high/high - low/high - high/low + low/low, with other exposures fixed.

Quantiles come from retained original training exposures and must lie in `[0, 1]`. Default grids span the observed range. Explicit grids must be finite, nonempty and strictly increasing. For an all-variable univariate call, a supplied `grid` is shared across exposures. `grids` contains one sequence for each surface axis. Calls are limited to 4096 output locations because joint prediction and contrast matrices are dense.

With `center=True`, curves and surfaces subtract the **all-median** reference profile, even if `fixed_quantile` is changed. This differs from R `bkmr` curve centering by the average fitted grid value. The pairwise interaction above also differs from the R summary that changes all background exposures together. These definitions should be specified when comparing results between packages.

Latent summaries use `h`. Response summaries use Gaussian/count means, binary probabilities or survival relative hazards, holding `X` constant and random intercepts at zero. Count summaries use zero log offsets (unit observation exposure). Survival summaries subtract relative hazards; they do not return survival-probability contrasts or hazard ratios. `X` is one raw adjustment vector; when omitted it defaults to retained training covariate means. `X` is rejected for latent summaries. Binary probabilities are transformed before contrasts are calculated. All contrasts use joint draws to preserve covariance and exactly cancel identical profiles.

Result objects contain `draws` with axes `(chains, selected_draws, locations)`, `mean`, `sd`, `lower`, `upper`, `locations`, `profiles`, `coefficients`, `scale`, `probability`, `kind` and `metadata`. `locations` identifies each variable, pair, quantile or grid value. `profiles` stores distinct original-unit prediction rows; `coefficients` maps their joint draws to reported summaries. `to_records()` returns location fields and numeric summaries. A `SurfaceResult` also contains `variables`, `grids` and `shape`; reshape its flat summaries to `shape`, with the second grid changing fastest.

These summaries describe conditional associations. Marginal quantile profiles can combine values rarely observed together. Credible intervals do not address poor exposure support, unmeasured confounding or model misspecification.

5.7 Plotting

Matplotlib is an optional dependency and is imported only when a plot is requested.

```
km.plot_trace(fit, parameter="lambda", indices=None, ax=None)
km.plot_pips(fit, ax=None)
km.plot_response(curve, ax=None)
km.plot_overall(effect, ax=None)
km.plot_surface(surface, ax=None)
km.plot_survival(prediction, row=0, ax=None)
```

Each function returns a Matplotlib axes object, creates one when `ax=None`, and leaves display, saving and global styles to the caller. `indices` selects a scalar component for a vector-valued trace parameter, for example `indices=0` for the first `beta` coefficient. Traces contain retained draws only. PIP plots require a selection fit.

`plot_response` accepts a univariate curve; `plot_overall` accepts an overall effect; `plot_surface` accepts a bivariate surface with at least two values on each grid. Curve and overall plots show pointwise posterior intervals. A surface shows the posterior mean only, with zero-centered colors when centered. Result objects retain its uncertainty. Request separate curves for exposures with different units.

5.8 Kernel helper

```
km.gaussian_kernel(Z1, Z2=None, r=None)
```

Returns a Gaussian kernel matrix between the supplied exposure rows. `Z2=None` uses `Z1` for both sides; `r=None` uses unit relevance for each column. Relevance values are nonnegative. This helper uses the values supplied directly and does not apply a fit's training transform.

5.9 Method sources

- Bobb et al. (2015), Bayesian kernel machine regression for estimating the health effects of multi-pollutant mixtures.
- Bobb et al. (2018), Statistical software for analyzing the health effects of multiple concurrent exposures via Bayesian kernel machine regression.
- Murray et al. (2010), Elliptical slice sampling.
- Stan count-distribution definitions.
- INLA manual, piecewise-constant proportional hazards (§3.15).

6 Checks and development

6.1 Run checks

```
python -m pip install -e ".[dev]"
python -m pytest
python -m ruff check src tests tools
python tools/check_public.py --history
python -m build
python -m twine check dist/*
```

CI builds a wheel and tests it outside the source checkout on Linux with Python 3.10 and 3.14, and Windows with Python 3.13. The minimum-dependency job pins NumPy 1.24.0, SciPy 1.10.0 and Matplotlib 3.7.0. See the repository's Actions results for the outcome on each commit; configuration alone does not establish that a platform passed.

6.2 Numerical evidence

Check	Purpose and limit
Independent kernel and conditional-normal calculations	Check covariance, prediction and joint contrasts
Installed R `bkmr` 0.2.2	Compare constructed saved states for Gaussian and probit models, clustered/unclustered conditional moments, PIPs and prior densities
Integrated Gaussian posterior	Compare sampled amplitude quantiles with one-dimensional quadrature after integrating out residual variance
Probit orthant posterior	Compare sampled amplitude quantiles with the analytic two-observation normal orthant likelihood
Uninformative selection case	Check the beta-binomial count distribution when exposure selection cannot change the kernel
Count and survival likelihoods	Compare Poisson/NB2 mass with SciPy and survival density/censoring terms with independent arithmetic and interval splitting
Elliptical slice posterior	Check a conjugate normal target
Poisson scalar posterior	Compare a constant GP posterior with one-dimensional integration
NB2 dispersion / survival baseline	Check conditional scalar/Gamma targets with other latent blocks held fixed; these are not joint-posterior studies
Legacy seeded behavior	Saved arrays agree bitwise with 0.1.0 in six Gaussian/probit selection cases
Diagnostic examples	Detect shifted chains, autocorrelation and constant traces
Structured likelihoods	Independent kernel/spectral, Gaussian block-density, conditional residual and censoring/CIF/path identities
Structured posterior targets	Constant-GP Gaussian, Poisson and delayed-entry survival targets against independent integration; amplitude held fixed for these checks
Finite-basis storage	Instrumented PP/RFF operations avoid training-size square matrices; this is not a large-dataset benchmark
Package and input checks	Exercise saved objects, preprocessing, errors, distribution contents and installed examples

The posterior-target tests use fixed seeds and Monte Carlo tolerances based on effective sample size. They are small correctness checks, not a repeated-dataset performance study. No general coverage, paper-study reproduction or complete R API equivalence is claimed.

6.2.1 Reproduce the R fixtures

R is needed only to regenerate fixtures. Use an isolated R library with `bkmr` **0.2.2**, `jsonlite` and the dependencies declared by that version of `bkmr` (including `fields`, `truncnorm` and `tmvtnorm`). Its CRAN archive has SHA256 `7ed29f74c4c78f3b8c663eac5ec417b4ea9bc586d789e8049e3ddc51541af9af`. Set `R_LIBS` to that library, then run:

```
Rscript tools/export_reference.R tests/fixtures/r_reference.json
Rscript tools/export_cluster_reference.R tests/fixtures/r_cluster_reference.json
python -m pytest tests/test_reference.py tests/test_clusters.py
```

The exporter verifies the installed version and records R/package versions. It calls installed `ComputePostmeanHnew` and `ExtractPIPs`, plus internal kernel and prior helpers. The fixtures contain **constructed states, not fitted R MCMC chains**. Preprocessing and intercept columns are supplied explicitly. Clustered fixtures use the installed public h-summary method and internal covariance helper. The R public summary API does not accept arbitrary

new-cluster labels; Python h+u prediction is checked with an independent block-normal calculation. Count and survival inference has no corresponding R `bkmr` family; its evidence is separate.

The elementwise tolerance is `rtol=2e-11, atol=2e-12`. R's exact summary uses sample covariance between conditional means (denominator $S-1$); Python's finite empirical mixture uses denominator S, where S is the retained state count. The comparison adjusts this convention explicitly. Sampling proposals and random-number streams differ; equal seeds do not produce matching chains.

6.3 Build the manual

The editable sources are the README and Markdown pages in `docs`. Install the documentation extra and a local `pdflatex` distribution:

```
python -m pip install ".[docs]"
python tools/build_manual.py
```

Pandoc and LaTeX build the PDF from Markdown. Intermediate TeX stays in a temporary directory. The small README example generates the response figure; its plotted values are also available in `docs/figures/response.csv`. The figure shows pointwise credible limits from short illustrative chains, not a performance benchmark.